

Московский физико-технический институт
Высшая школа программной инженерии

Формальные языки и сложность вычислений
3 семестр, 2026/2027 уч. год

Лекция 1
Теорема об иерархии по времени.
Универсальная МТ. Классы сложности.
Классы P и NP

Лектор: Зухба Анастасия Викторовна
Автор: [Консультант по борьбе с продуктивностью](#)

1 Блок воспоминаний

Итак, если говорят "язык распознаётся за..." то без дополнительных оговорок имеют в виду многоленточную ДМТ с лентами, бесконечными в обе стороны.

Определение 1.1. Классом $\text{DTIME}(T(n))$ называется класс языков, которые распознаются за время $O(T(n))$.

Обычно в качестве $T(n)$ рассматривают только "хорошие" функции: $T(n) \geq n$, монотонна и её "можно быстро посчитать".

Определение 1.2. Функция $T: \mathbb{N} \rightarrow \mathbb{N}$ называется **конструируемой по времени**, если существует алгоритм, который за время $O(T(n))$ по 1^n (то есть числу в унарной записи) получает $T(n)$ (в бинарной записи).

2 Детали модели

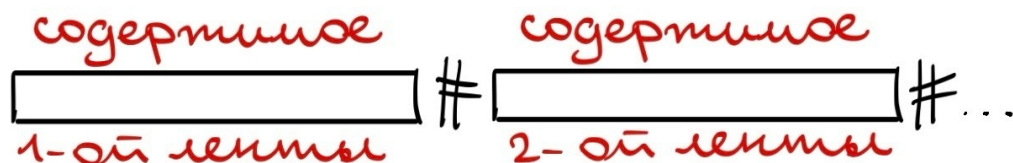
Теорема 2.1 (Теорема о линейном ускорении). Пусть язык $\mathcal{L}$ разрешается машиной M , которая работает за время T . Тогда для любого $c > 0$ существует машина M' , эквивалентная M и работающая за время T' , где

$$T'(n) = cT(n) + n + 100$$

Теорема 2.2 (Малый алфавит). Пусть ДМТ M работает за время T и имеет алфавит A . Пусть входной и выходной алфавиты M равны $\{0, 1\}$. Тогда существует эквивалентная её ДМТ M' с алфавитом $\{0, 1, \lambda\}$, которая работает за время $O(\log |A| \cdot T(n))$.

Теорема 2.3 (От нескольких лент к одной). Любой язык, который можно распознать за время $O(T(n))$ на многоленточной машине Тьюринга, можно распознать за время $O(T^2(n))$ на одноленточной машине.

Доказательство. Опишем лишь **идею** доказательства. Пусть k -ленточная ДМТ M распознаёт язык $\mathcal{L}$ за время $O(T(n))$. Рассмотрим моделирующую её одноленточную ДМТ M' (такая прелесть существует по теореме прошлого семестра), которая устроена вот как: её единственная и неповторимая лента условно разделена на k блоков по $T(n)$ ячеек каждый, которые отделяются некоторым спец символом, который не содержался в алфавите M (к примеру, решёткой, как на рисунке ниже).



Соответственно в i -ом блоке мы поддерживаем содержимое i -ой ленты M на каждом такте работы M' . Заметим, что машина M' в течение одного такта работала на каждой ленте на более, чем $T(n)$ времени. Следовательно, в процессе моделирования машиной M' работы M на каждой ленте за один такт, она не будет выходить за текущий блок и портить решётки. При этом время моделирования одного такта есть $O(k \cdot (T(n) + 1)) = O(T(n))$. Так как тактов всего $T(n)$, получаем то, что нужно. $\square$

Замечание 2.1 (Односторонняя и двухсторонняя ленты). Лента, бесконечная в обе стороны, моделируется на односторонней ленте при помощи расширения алфавита. Если же речь о многоленточной МТ, то лента, бесконечная в обе стороны, тривиально моделируется на двух односторонних лентах. Моделирование линейно по $T(n)$.

3 Теорема об иерархии по времени

Теорема 3.1 (Об иерархии по времени). Пусть $f: \mathbb{N} \rightarrow \mathbb{N}$, $g: \mathbb{N} \rightarrow \mathbb{N}$ — строго монотонные функции, конструируемые по времени, такие, что:

$$f(n) \log f(n) = o(g(n))$$

Тогда

$$\text{DTIME}(f(n)) \subsetneq \text{DTIME}(g(n))$$

Доказательство. **Идея** доказательства состоит в том, чтобы перебирать все возможные алгоритмы, работающие не больше $O(f(n))$, и постепенно строить язык так, чтобы все эти алгоритмы ошибались. При этом само построение работает за время, не сильно большее, чем $f(n)$. $\square$

4 Универсальная МТ

С прошлого семестра нам известно о существовании универсальной машины Тьюринга U . То есть для любой машины Тьюринга M и любых слов x, y таких, что $M(x) = y$, выполнено $U(\langle M \rangle, \langle x \rangle) = \langle y \rangle$.

Теперь нас интересует время работы, потому заранее договоримся, что рассматриваем сейчас именно детерминированные машины Тьюринга.

Теорема 4.1. Существует универсальная **одноленточная** ДМТ U_1 такая, что если M работает за $O(T(n))$, то U_1 работает при моделировании M за $O(T^2(n))$.

Теорема 4.2. Существует U — универсальная машина Тьюринга (заметим, что здесь уже не обязательно одноленточная) такая, что если МТ M работает за время $T(n)$, то U при моделировании M работает за время

$$cT(n) \log(T(n)) + O(1)$$

Здесь c зависит только от $\langle M \rangle$, а $n = |x|$.

5 Распознавание на НМТ

Рассмотрим недетерминированную МТ. Единственным её отличием является неоднозначность функции (таблицы) переходов δ (то есть $\delta: A \times Q \rightarrow A \times \{-1, 0, +1\} \times Q$).

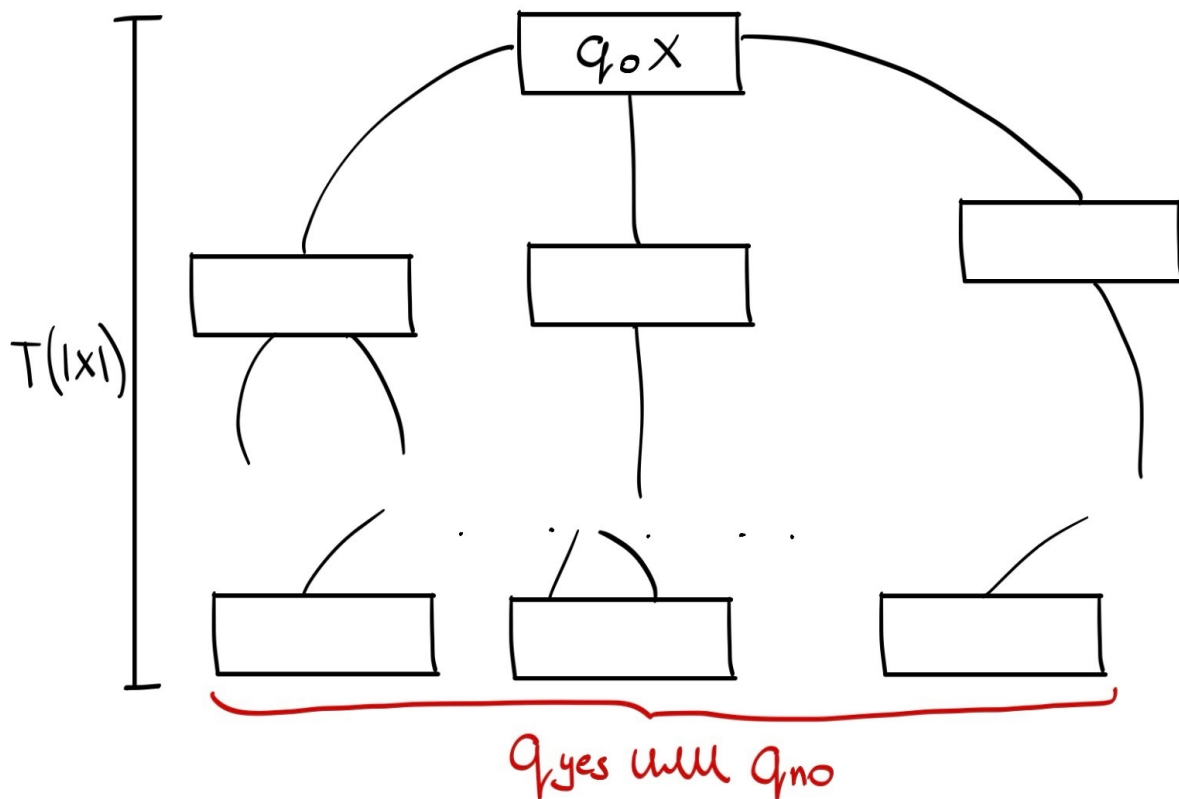
Пример 5.1. В ячейке неоднозначной таблицы переходов может содержаться несколько вариантов переходов (несколько слов):

$$\delta(a, q) = \left\{ \begin{array}{l} (a, +1, q_2) \\ (b, -1, q_3) \\ (c, 0, q_1) \end{array} \right\}$$

Недетерминированная машина M распознаёт язык $\mathcal{L}$ за время $T(n)$, если при всех $x \in \{0, 1\}^*$ верно следующее:

- Любое вычисление, начинающееся с x , не длиннее $T(|x|)$ (иначе говоря, машина в любом случае останавливается не более, чем за $T(|x|)$ шагов)
- Если $x \in \mathcal{L}$, то **найдётся** вычисление, начинающееся с x и заканчивающееся в состоянии q_{yes} .
- Если $x \notin \mathcal{L}$, то **любое** вычисление, начинающееся с x , заканчивается в состоянии q_{no} .

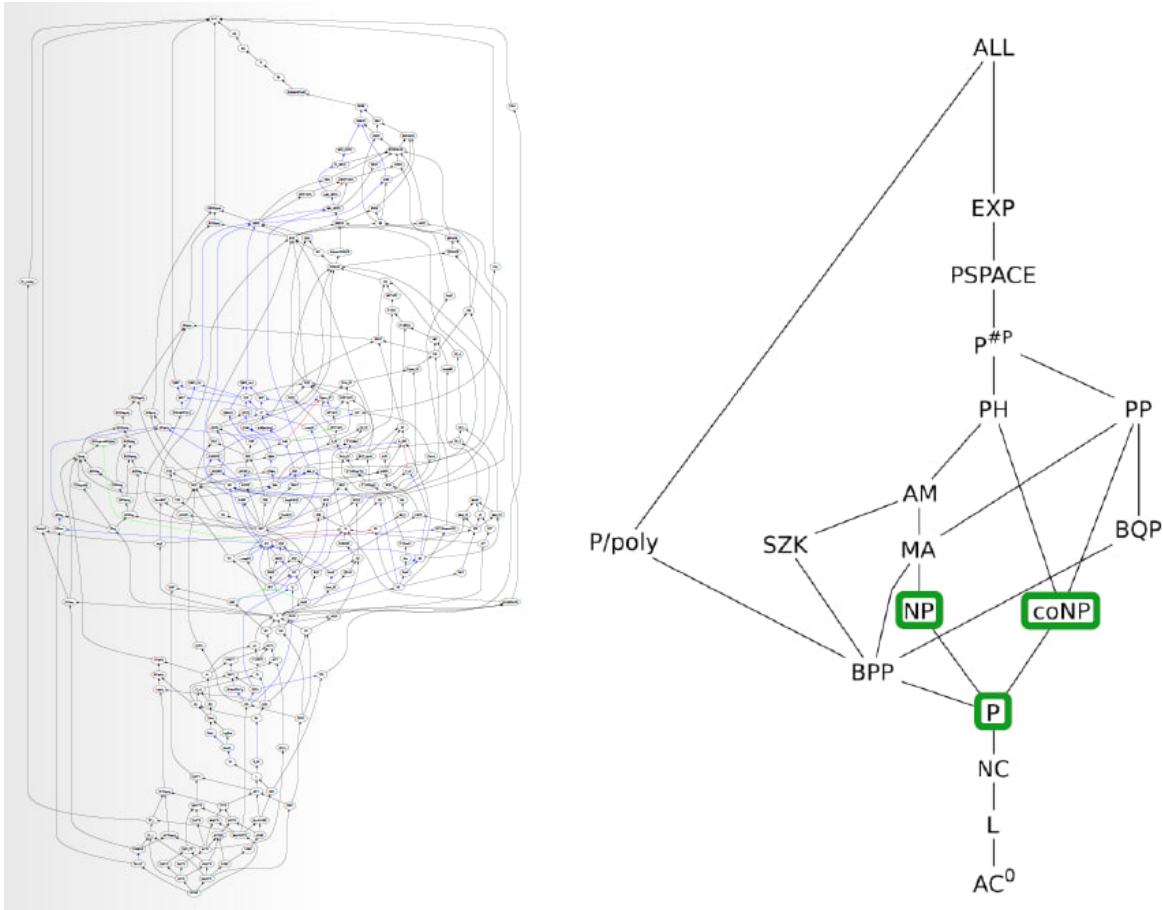
Теперь поясним на рабоче-крестьянском. Любое вычисление можно представить как путешествие по дереву конфигураций МТ, где в корне сидит начальная конфигурация q_0x , в листьях — финальные в состояниях q_{yes} или q_{no} , а спуск вниз делается при помощи отображения на конфигурациях МТ.



Выходит, что машинка, распознающая язык, для любого слова x может спуститься не более, чем $T(|x|)$ раз прежде, чем зайдёт в принимающее состояние. Тогда высота дерева в принципе не превышает $T(|x|)$.

6 Классы сложности

Для затравки приведём две страшные картинки: левая описывает все (или почти все) известные классы сложности, а правая — те, смысл рассмотрения которых для нас чуть больше остальных. В частности, зелёным обведены те, рассмотрением которых мы займёмся в самое ближайшее время.



6.1 Класс P

Определение 6.1. Классом P называется множество языков, распознающихся за полиномиальное время

$$P = \bigcup_{c=1}^{\infty} DTIME(n^c) = DTIME(\text{poly}(n))$$

Пример 6.1. Приведём пример языков, лежащих в классе P:

- $GCD = \{(a, b, d) \mid \text{число } d \text{ является наибольшим общим делителем чисел } a \text{ и } b\}$
- $PATH = \{(G, s, t) \mid \text{в графе } G \text{ есть путь из } s \text{ в } t\}$
- $EULERCYCLE = \{G \mid \text{в графе } G \text{ есть эйлеров цикл}\}$
- $BIPARTITE = \{G \mid \text{граф } G \text{ двудольный}\}$

6.2 Класс NP

Определение 6.2. Классом $NTIME(T(n))$ называется множество языков, распознаваемых на недетерминированной машине Тьюринга за время $O(T(n))$.

$$NP = \bigcup_{c=1}^{\infty} NTIME(n^c)$$

В названии класса N обозначает "недетерминированные вычисления", а P — "полиномиальное время".

Теорема 6.1. Пусть функция T конструируема по времени. Тогда

$$\text{NTIME}(T) \subseteq \text{DTIME}(2^{O(T)})$$

Доказательство. Обозначим **идею** доказательства. Пусть k -ленточная НМТ M разрешает язык $\mathcal{L}$ за время $T(n)$. Построим $(k+1)$ -ленточную ДМТ M' , разрешающую $\mathcal{L}$ за $d^{O(T(n))}$, где d — максимальное число слов в ячейке таблицы переходов M , или же максимальное ветвление в дереве конфигураций M . Понятно, что $d \leq |A| \cdot 3 \cdot |Q|$, то есть для фиксированной машинки это константа (не зависит от размера входа).

Мы помним, что у НМТ дельта неоднозначная, а детерминированная машинка хочет точно знать, куда идти на очередном такте. Давайте же на $(k+1)$ -ой ленте поддерживать слово длины $T(n)$ (длина такая, потому что определение распознавания языка НМТ), по которому при очередном вычислении (спуска до листа) будет идти ДМТ M' и по написанной цифре в ячейке точно понимать, какое слово из множества в ячейке брать. Тогда вычисление от x до листьев будет однозначным, а время работы для одного спуска $O(T(n))$. После вычисления мы должны, конечно, поменять слово на служебной $(k+1)$ -ой ленте на следующее в лексикографическом порядке, чтобы рассмотреть новый путь в дереве. Всего различных путей ясен пенёк не больше $d^{T(n)}$. Если в процессе спуска выяснилось, что мы должны взять несуществующее слово (так может случиться, ведь мы перебираем как бы все числа в d -ричной системе счисления по возрастанию, а какому то из них может не соответствовать пути, ведь мы делаем оценку сверху), или мы дошли до листа, но там состояние q_{no} , то просто меняем слово на служебной ленте на следующее в лексикографическом порядке и запускаемся сначала в поисках счастья. Если на нижнем этаже оказалось состояние q_{yes} , то это победа, и ДМТ M' завершает трудовой день. $\square$

Гипотеза 1.

$$\text{P} \neq \text{NP}$$

Пример 6.2. Приведём пример языков, лежащих в классе NP:

- Все задачи из класса P
- $\text{SAT} = \{\varphi \mid \varphi \text{ — выполнимая булева формула}\}$
- $\text{CLIQUE} = \{(G, k) \mid \text{в графе } G \text{ найдётся полный подграф хотя бы на } k \text{ вершинах}\}$
- $3\text{COL} = \{G \mid \text{вершины графа } G \text{ можно правильно раскрасить в 3 цвета}\}$
- $\text{MAXCUT} = \{(G, k) \mid \text{вершины графа } G \text{ можно разбить на два множества } S \text{ и } T \text{ так, что между частями проведено не менее } k \text{ рёбер}\}$

Приведём теперь эквивалентное определение класса NP, с помощью которого мы лучше поймём суть доказательства выше.

Определение 6.3. Классом NP называется множество языков $\mathcal{L}$, для которых существует функция $V(x, s)$ с булевыми значениями, вычисляемая за полиномиальное время от длины **первого аргумента** такая, что:

- Если $x \in \mathcal{L}$, то $\exists s V(x, s) = 1$
- Если $x \notin \mathcal{L}$, то $\forall s V(x, s) = 0$

Второй вход s часто называют **сертификатом**, а функцию V — **верификатором**. Таким образом, сертификат удостоверяет, что $x \in \mathcal{L}$, а верификатор проверяет верность сертификата.